National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	07
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	App Development (Cont.) Backend-Database Integration
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

Application Development

Application Development refers to the process of designing, creating, testing, and maintaining software applications to solve specific problems or perform particular tasks.

In the current and upcoming labs, Application Development focuses on using **PHP** as the server-side programming language together with **MySQL** as the database management system to build a complete database system in the form of a dynamic web application.

1 Application-Database Interaction

Before an application can work with a database, there must be a **connection layer** between the application and the DBMS.

Key Points:

- **Database Server**: MySQL runs as a server process.
- **Client Application**: PHP scripts act as clients sending queries.
- **Connection**: PHP uses a connection object to communicate with MySQL.
- **Benefits of Proper Connection Handling**:
    - Securely send/receive data
    - Handle errors gracefully
    - Reuse connections efficiently

Flow Diagram (Simplified):

PHP Application -> PDO -> MySQL Server -> Tables -> Return Data

- The application sends SQL queries via PDO (or mysqli).
- The database executes the query.
- Results are returned to the application.

1.1 PHP Data Objects (PDO)

PDO is a **database abstraction layer** in PHP:

- Provides a consistent API for multiple DBMS (MySQL, PostgreSQL, SQLite, etc.)
- Encourages **secure database interaction** using prepared statements
- Allows **exception handling** for errors

- Supports **fetching results in multiple modes** (associative, numeric, both)

Benefits:

Feature	Description
Multi-DBMS	Can connect to different database systems
Prepared Statements	Prevents SQL injection, safer queries
Error Handling	PDOException helps catch errors cleanly
Flexible Data Fetching	Associative arrays, numeric arrays, objects

1.2 Connecting to MySQL using PDO

1.2.1 PDO Connection Example

```
<?php

$host = 'localhost';
$db = 'lab7'
$user = 'root';  // DB username
$pass = '';      // DB password

$dsn = "mysql:host={$host};dbname={$db}";  // Data Source Name

try {
    $pdo = new PDO($dsn, $user, $password);
    print 'Connected successfully!';

    // Set error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Set default fetch mode to associative array
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);

} catch (PDOException $e) {
    die("Connection Failed: {$e->getMessage()}");
}
```

```
?>
```

1.2.2 Key Parameters

Parameter	Purpose
`$dsn`	A string specifying database type, host, database name
`$user`	DBMS username
`$pass`	DBMS password
`ERRMODE_EXCEPTION`	Throws exceptions on database errors for easier debugging
`FETCH_ASSOC`	Fetches query results from database as associative arrays

1.3 PDO Functions for CRUD Operations

1.3.1 `exec()`

- Executes an SQL statement **without returning a result set** (INSERT, UPDATE, DELETE)
- Returns **number of affected rows**

Examples:

```
$rows = $pdo->exec("UPDATE students SET name = 'Umar' WHERE id = 5");
echo "{$rows} rows updated.";

$rows = $pdo->exec("DELETE FROM students WHERE id = 5");
echo "{$rows} rows deleted.";
```

1.3.2 `query()`

- Executes **SQL SELECT** statements (queries)
- Returns a **PDOStatement object** which can contain either the query result set or a boolean value (true/false) depending on the type of SQL statement.

Examples:

```
// Fetch all students
$stmt = $pdo->query("SELECT * FROM students");
```

```
$students = $stmt->fetchAll();  // Returns associative array

// Fetch the top 3 highest CGPA students
$stmt = $pdo->query("SELECT * FROM students ORDER BY cgpa DESC LIMIT 3");
$top3_students = $stmt->fetchAll();
```

1.3.2 `prepare()` & `execute()`

- Used for **prepared statements** instead of raw SQL statements (DDL, DML, DQL etc).
- Prevents SQL injection by binding parameters

Examples:

```
$stmt = $pdo->prepare("INSERT INTO students (name, cgpa) VALUES
(:name, :cgpa)");

$stmt->execute([
    ':name' => 'Ali',
    ':cgpa' => 2.8
]);
```

Advantages of Prepared Statements:

Advantage	Explanation
Security	Parameters are automatically escaped
Reusability	Prepare once, execute multiple times
Performance	Optimized for repeated execution

1.4 Organizing PHP Code

Construct / Function	Description
`include`	Includes a file, produces warning if file missing
`include_once`	Includes a file once, avoids multiple inclusions
`require`	Includes a file, produces fatal error if missing

`require_once`	Includes a file once, produces fatal error if missing

Example:

- Separate DB Credentials Code:

```
// db_config.php

<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'lab7');
define('DB_USER', 'root');
define('DB_PASS', '');

define('MYSQL_DSN', 'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME);
?>
```

- Separate DB Connection Code:

```
// db_connection.php

<?php
require_once 'db_config.php';

try {
    $conn = new PDO(MYSQL_DSN, DB_USER, DB_PASS);
    print 'Connected Successfully!';

    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);

} catch (PDOException $e) {
    die("Connection Failed: {$e->getMessage()}");
}
```

1.5 Why Try-Catch is Essential for DB Queries

- **Database errors happen frequently**: wrong query, constraint violation, server down

- Wrapping queries in **try-catch** ensures:
  - Proper debugging
  - Application doesn't crash
  - Errors can be logged
  - Secure handling of sensitive info

Example:

```
try {
    $stmt = $pdo->query("SELECT * FROM non_existing_table");
} catch (PDOException $e) {
    echo "Query failed: {$e->getMessage()}";
}
```

2 PHP Superglobals

Superglobals are **built-in PHP variables** (associative arrays) that:

- Are always available (no need to declare them)
- Can be accessed from anywhere in the script
- Store important data about:
  - Forms
  - URLs
  - Sessions
  - Cookies
  - The server itself

They are called **"super" globals** because they work everywhere in your code. They are heavily used when connecting frontend forms to backend scripts.
Some of the most useful superglobals are listed below:

Superglobal	Purpose	User Visibility	Common Use
`$_GET`	Get data from URL	Yes	Search, filters
`$_POST`	Get form data	No	Login, forms
`$_SESSION`	Store user session data	No	Login system
`$_COOKIE`	Store data in browser	Yes	Preferences

$_SERVER	Server & request info	Partially	Debugging